

Thinking, Fast and Slow for Arithmetic: A Dual-Model Agent with Memory for Elementary Multiplication

Benny Signer
Undergraduate, Computer Science

September 2026

Abstract

Large reasoning models solve hard problems reliably but are slow and expensive; small instruction-tuned models are fast and cheap but unreliable on anything non-trivial. Inspired by the two-systems account of cognition in Daniel Kahneman’s *Thinking, Fast and Slow*, we build an agentic system that routes each problem to the cheapest resource likely to solve it correctly. A fast, non-reasoning “System 1” model answers what it is confident about and *escalates* the rest, through native tool calls, to a slow, expensive “System 2” reasoning model; a deterministic exact-match memory is consulted before either model runs. We evaluate the design on a stream of 200 elementary multiplication problems (with repeats) across four configurations. Confidence framed routing alone reaches 99.5% accuracy while sending only 15.5% of problems to the reasoning model, at 43% of the reasoning-only cost. Adding memory serves 60% of the stream at zero token cost and drives the full agent to roughly $6.3\times$ lower cost and $6.5\times$ lower mean latency than the reasoning-only baseline, for a 0.5 percentage-point accuracy cost (98.5% vs. 99.0%). We report the central trade-off honestly: memory couples correctness to the quality of the first-contact routing decision, so a confident early mistake propagates to every repeat. This is an exploratory undergraduate white paper; we draw modest conclusions and outline where the design should be pushed next.

1 Introduction

In *Thinking, Fast and Slow*, [Kahneman \(2011\)](#) describes human thought as the interaction of two systems: a fast, automatic, low-effort *System 1* that handles the routine, and a slow, deliberate, effortful *System 2* that is recruited only when a problem is hard enough to demand it. Crucially, System 1 does not attempt everything; it recognizes when a question exceeds its reach and hands it off. Much of the efficiency of human cognition comes from this routing—most of what we do never troubles the expensive machinery at all.

Contemporary language-model deployments face a structurally similar situation. A small instruction-tuned model is cheap and fast but error-prone; a large reasoning model is accurate but slow and costly. Using the reasoning model for *every* query is the machine analogue of engaging effortful deliberation for $2 + 2$: correct, but wasteful. This paper asks a simple question:

Can a small model act as a System 1 that decides which problems it can answer and which it should defer to a larger System 2, and does adding a memory of previously solved problems yield a system that is substantially cheaper and faster than always deliberating, without giving up much accuracy?

We study this in the deliberately narrow and verifiable domain of elementary multiplication, where correctness is unambiguous and difficulty is easy to control. Our contribution is not a new model or a new routing algorithm, but an honest end-to-end measurement of a Kahneman-inspired agent—which we call **TFaS** (Thinking, Fast and Slow)—built from off-the-shelf models, and a frank account of where memory helps and where it hurts. We deliberately keep the claims small: this is an exploratory study on a single toy task, intended to establish the shape of the trade-offs rather than to settle them.

2 Related Work

Dual-process theory. The framing of cognition as two interacting systems—an intuitive, fast System 1 and a reflective, slow System 2—is popularized by [Kahneman \(2011\)](#). We borrow the metaphor as an engineering design principle rather than a cognitive claim: our “System 1” and “System 2” are simply two models with very different cost/accuracy profiles and an explicit handoff between them.

LLM cascades and routing. The idea of using a cheap model first and escalating only when necessary is developed by [Chen et al. \(2023\)](#) in FrugalGPT, which studies LLM “cascades” that query a sequence of models and stop as soon as an answer is deemed good enough, reducing cost while preserving or improving quality. Our design is a small, concrete instance of this family with two differences of emphasis: (i) the escalation decision is made by the small model itself through a native tool call framed around confidence, rather than by a separate scoring model, and (ii) we add a deterministic memory checked before any model runs, so repeated problems cost nothing. We do not claim novelty over the cascade literature; we treat this work as a hands-on replication of the core idea in a controlled arithmetic setting, with memory added.

3 System Architecture

The full TFaS agent is composed of three components—a memory, a fast System 1, and a slow System 2—connected by a fixed control flow. Both models are served through the OpenRouter API at temperature 0.

Memory. Memory is a deterministic exact-match cache keyed by the ordered problem string (e.g. “7x8”). It is consulted *before* any model call: a hit returns the stored answer immediately, consuming zero tokens. It is written back after every model-produced answer, whether that answer came from System 1 or System 2. Because the key is the exact ordered string, the cache makes no attempt to exploit commutativity ($a \times b = b \times a$) or any other structure; it only recognizes literal repeats.

System 1 (fast). System 1 is `mistralai/ministral-3b-2512`, a 3-billion-parameter, non-reasoning instruction model ([Mistral AI, 2024](#)). Given a problem, it must respond with one of two *native tool calls*: `answer(result)` to commit to an integer answer, or `escalate()` to defer to System 2. The prompt is *confidence-framed*: the model is instructed to answer only when it is highly confident the result is exactly correct, and otherwise to escalate rather than guess, on the explicit principle that “it is much better to escalate than to give a wrong answer.” The routing decision is thus genuine self-assessment by the model, not a hard-coded rule on the number of digits.

System 2 (slow). System 2 is `deepseek/deepseek-r1-0528`, a larger reasoning model (DeepSeek-AI, 2025). It reasons step by step and then emits a line of the form `ANSWER: <integer>`, from which the final answer is parsed. It is configured with OpenRouter throughput provider routing and a 6000-token budget.

Control flow. The agent’s `solve()` procedure ties the three components together, as shown in Algorithm 1: check memory; on a miss, ask System 1; if System 1 escalates, invoke System 2; then cache the model-produced result. Removing components recovers the baselines we compare against in Section 4: dropping memory and forcing every problem to System 2 gives the reasoning-only baseline, and using only System 1 gives the cheap baseline.

Algorithm 1 `solve(problem)` — the full TFaS agent

Require: problem string p (e.g. "384x279")

```

1: if  $p \in \text{MEMORY}$  then                                ▷ zero tokens on a hit
2:   return  $\text{MEMORY}[p]$ 
3: end if
4:  $(\text{action}, r) \leftarrow \text{SYSTEM1}(p)$                     ▷ native tool call: answer or escalate
5: if  $\text{action} = \text{answer}$  then
6:    $a \leftarrow r$ 
7: else                                                    ▷ action = escalate
8:    $a \leftarrow \text{SYSTEM2}(p)$                             ▷ step-by-step reasoning
9: end if
10:  $\text{MEMORY}[p] \leftarrow a$                                 ▷ write back for future repeats
11: return  $a$ 

```

4 Experimental Setup

Dataset. We generate a unique pool of 80 multiplication problems: 45 *easy* (single-digit $\times$ single-digit, factors 2–9) and 35 *hard* (at least one factor with 2–3 digits, a mix of $2d \times 1d$, $2d \times 2d$, $3d \times 1d$, and $3d \times 2d$). From this pool we draw a *stream of 200 problems with repeats*, of which 120 (60%) are repeated occurrences; the stream contains 116 easy and 84 hard occurrences. Generation is seeded (seed 0) for reproducibility. The presence of repeats is what makes memory meaningful, and is intended to loosely mimic the recurring, over-learned character of the arithmetic facts that human System 1 answers automatically.

Configurations. We run four configurations over the identical stream:

- **pure_system1** — System 1 only, no memory. It must answer every problem itself.
- **pure_system2** — System 2 only, no memory. Every problem is deliberated on.
- **hybrid_no_memory** — System 1→System 2 routing, no memory. This isolates the value of routing alone.
- **hybrid (TFaS)** — the full agent: memory →System 1→System 2.

Metrics. For each configuration we record accuracy (overall and by tier), per-problem model latency, wall-clock time, total tokens, USD cost (taken from OpenRouter’s own accounting), the escalation rate (fraction of problems routed to System 2), the memory hit rate, and the number of System 2 calls.

Infrastructure and concurrency. All models run through OpenRouter at temperature 0, and all four runs completed with zero errors. The three stateless configurations were run concurrently with 12 workers, because System 2 (DeepSeek-R1) exhibits heavy-tailed latency—individual calls ranged from about 12 to 113 seconds—and serial execution would have been needlessly slow. The memory-enabled hybrid, by contrast, was run *sequentially*: cache write-backs must be visible to later repeats in the stream for memory to function as intended. This asymmetry is why the full hybrid’s wall-clock time (Table 1) is not comparable to the concurrent runs’ wall-clock time and should be read alongside total latency instead.

5 Results

Table 1 summarizes the four configurations on the curated metrics that matter most; the complete auto-generated table with all columns appears in Appendix A (Table 2).

Table 1: Main results: accuracy (overall / easy / hard), mean per-problem latency, total tokens, cost, escalation rate, and memory hit rate for each configuration over the 200-problem stream. The full 15-column table is in Appendix A.

config	acc.	easy	hard	lat. (s)	tokens	cost (\$)	esc. / mem.
pure_system1	0.920	1.000	0.810	0.53	26,113	0.0026	0.000 / 0.000
pure_system2	0.990	1.000	0.976	24.14	237,570	0.5282	1.000 / 0.000
hybrid_no_memory	0.995	1.000	0.988	9.39	152,579	0.2272	0.155 / 0.000
hybrid (TFaS)	0.985	1.000	0.964	3.74	57,644	0.0835	0.060 / 0.600

5.1 Routing alone already pays off

The clearest result is that confidence-framed routing, with no memory at all, essentially matches the reasoning-only model at a fraction of its cost. `hybrid_no_memory` reaches **99.5%** overall accuracy—at or slightly above `pure_system2`’s 99.0%—while sending only **15.5%** (31 of 200) of problems to System 2. Cost falls to \$0.2272, about **43%** of `pure_system2`’s \$0.5282 (Figures 2 and 5). In other words, System 1 correctly recognized which problems it could handle—all of the easy tier and most of the hard tier—and deferred the remainder to the model that could.

5.2 Memory is the large multiplier

Adding memory turns a good cost reduction into a large one. Because 60% of the stream consists of repeats, the full `hybrid` answers **60%** of problems directly from cache at zero token cost, escalates only **6%** (12 of 200) to System 2, and costs **\$0.0835**. Relative to `pure_system2` this is roughly 6.3× **cheaper**, uses about **24%** of the tokens (57,644 vs. 237,570), and has 6.5× **lower** mean latency (3.74s vs. 24.14s), for a **0.5 percentage-point** accuracy cost (98.5% vs. 99.0%). The accuracy–cost picture is summarized in Figure 6, and the divergence of the cumulative-cost curves over the stream in Figure 7: the memory-enabled curve flattens as repeats begin to be served from cache, while the reasoning-only curve climbs linearly.

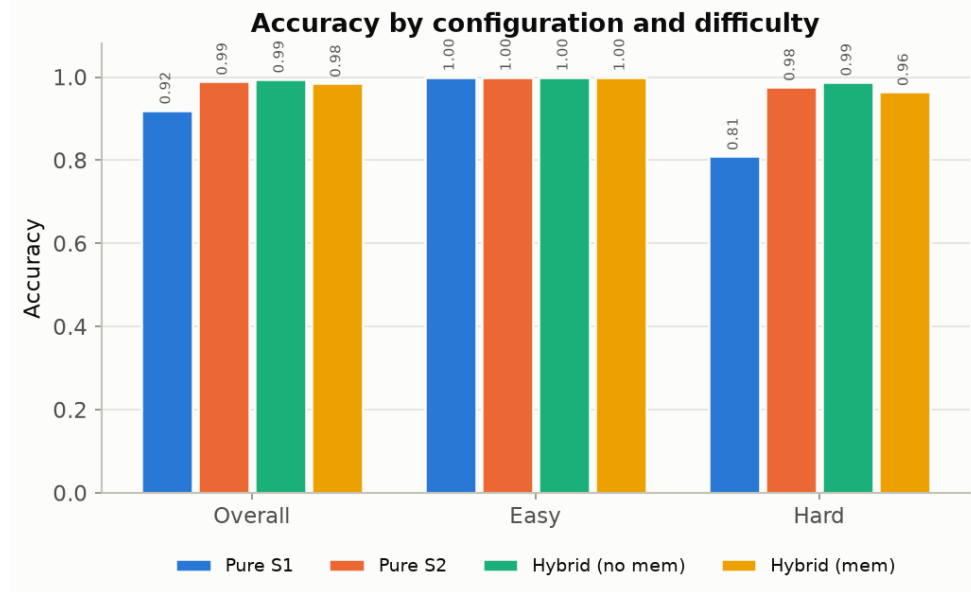


Figure 1: Accuracy by configuration and difficulty tier. All configurations are perfect on the easy tier; the differences live entirely in the hard tier, where `pure_system1` drops to 0.81.

5.3 The extremes

The two pure baselines frame the trade space. `pure_system1` is about $203\times$ **cheaper** and $46\times$ **lower** in mean latency than `pure_system2`, but only **92.0%** accurate—and its errors are concentrated exactly where one would expect, in the hard tier, where it misses 19% of problems (81.0% accuracy) despite being perfect on the easy tier (Figure 1). `pure_system2` buys back that hard-tier accuracy (97.6%) at two orders of magnitude more cost and latency (Figures 4 and 3). The hybrids sit in between, and much closer to the cheap corner than to the expensive one.

6 Discussion

Memory’s double edge. The most important nuance in Table 1 is that the full **hybrid** (98.5%) is slightly *below* **hybrid_no_memory** (99.5%), even though they share the same router. The cause is structural. Memory caches System 1’s decision on a problem’s *first* occurrence. If System 1 is confidently *wrong* on a hard problem the first time it appears—committing an answer instead of escalating—that wrong answer is written to the cache and then replayed for every subsequent repeat, which never gets a fresh chance to escalate to System 2. Memory faithfully amplifies the first decision, correct or not. The effect is compounded by mild provider-side nondeterminism at temperature 0: without memory, a borderline hard problem can occasionally “re-roll” into a correct escalation on a later occurrence, an escape hatch that memory removes by construction. This is the central trade-off of the design: memory buys large efficiency gains but couples end-to-end correctness to the quality of first-contact routing.

Small models need permission to say “I don’t know.” An early calibration experiment shaped the whole design. With a naive prompt, the 3B System 1 was *overconfident*: asked for 384×279 , it answered 106536—which is wrong; the correct value is 107136—instead of escalating. The model would rather guess than admit uncertainty. Reframing the prompt around confidence

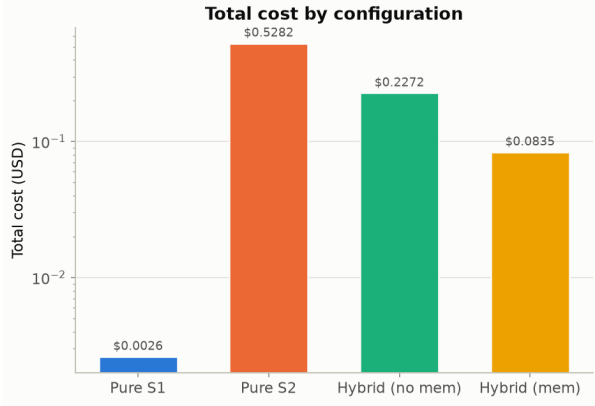


Figure 2: Total USD cost per configuration (Open-Router accounting).

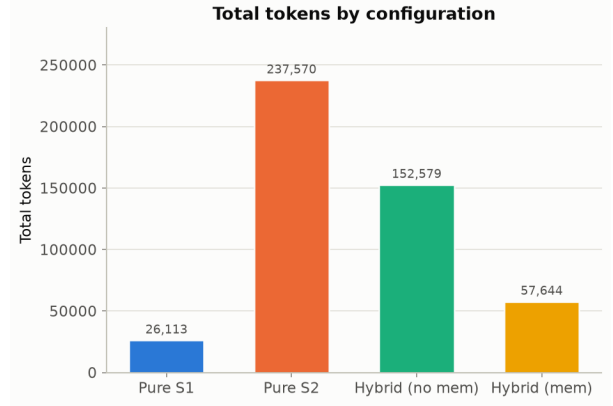


Figure 3: Total tokens consumed per configuration.

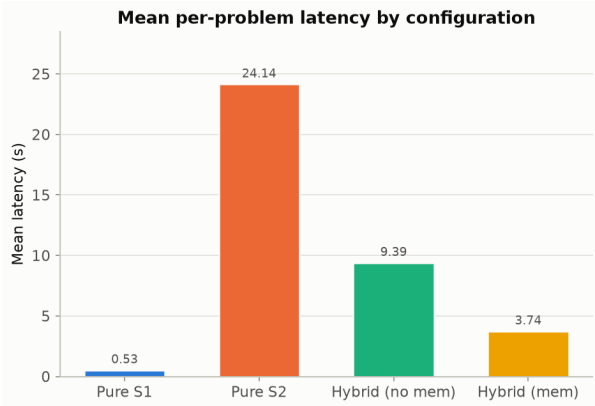


Figure 4: Mean per-problem model latency per configuration (seconds).

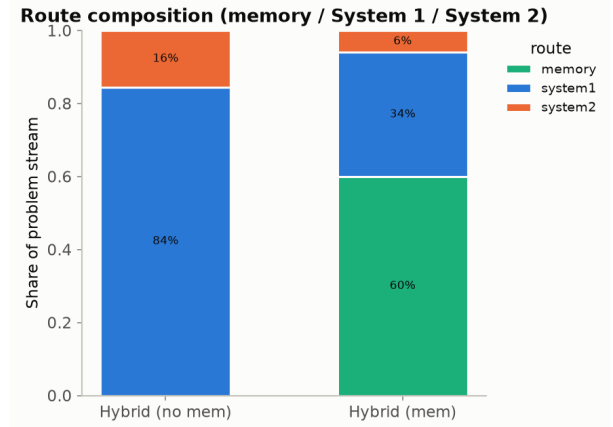


Figure 5: Route composition for the two hybrid configurations: the share of the stream served by memory, System 1, and System 2.

and deference (“it is much better to escalate than to give a wrong answer”) fixed the routing behavior: the same model then answered 6×7 and 12×8 but escalated 23×47 and 384×279 . The lesson is that a small model’s willingness to defer is not automatic; it has to be explicitly granted. This mirrors the intuition from [Kahneman \(2011\)](#) that the value of System 1 lies as much in knowing its limits as in its speed.

Latency tail. System 2’s latency was heavy-tailed—individual calls ranged from roughly 12 to 113 seconds—which is why the reasoning-only mean latency (24.14s) is so much larger than either hybrid’s. Every problem routed away from System 2 avoids exposure to this tail, so the latency benefit of routing and memory is even larger in the worst case than the means suggest. It also motivated our infrastructure choice to run the stateless configurations concurrently while keeping the memory hybrid sequential (Section 4).

7 Limitations and Future Work

This is an exploratory study, and its limitations are real.

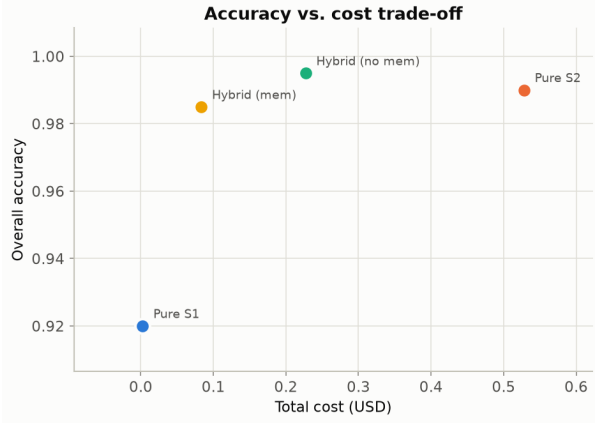


Figure 6: Accuracy versus cost—the core trade-off. The hybrids occupy the desirable upper-left region.

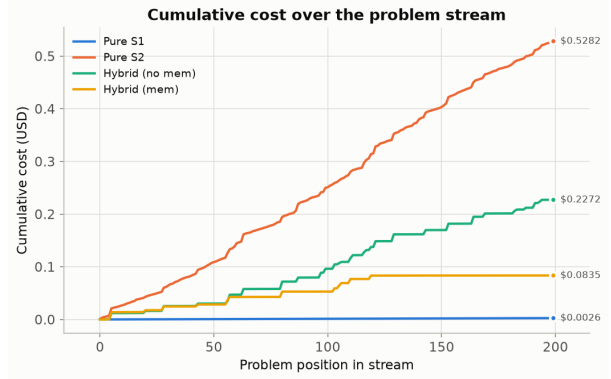


Figure 7: Cumulative cost over the problem stream. The memory-enabled hybrid flattens as repeats are served from cache.

- **Single, toy task.** We study only elementary multiplication, where answers are exactly verifiable and difficulty is trivial to control. Real deployments involve open-ended tasks where “correctness” and “difficulty” are far harder to define, and System 1’s self-assessment may not transfer.
- **Exact-match memory.** The cache keys on the literal ordered string and so ignores commutativity (7×8 and 8×7 are distinct keys) and generalizes to nothing beyond exact repeats. A semantic or approximate memory could capture far more reuse.
- **Modest scale.** A single stream of $N = 200$ over a pool of 80 is small; the 0.5-point gaps between the strong configurations are within the range one would want to confirm with repeated runs and larger streams.
- **Provider variance.** Temperature 0 was not perfectly deterministic across provider routing, which contributes noise to the comparisons (Section 6) and complicates exact reproduction.
- **No fine-tuning.** Both models are used off the shelf; System 1’s routing quality was shaped only by prompting.

Natural next steps include a *learned router* (rather than relying on the small model’s self-report), a *semantic or approximate memory* that exploits commutativity and near-duplicates, extension to *harder task domains* where verification is nontrivial, and explicit *confidence calibration* of System 1 so that first-contact routing—which we show is the factor memory most amplifies—is as reliable as possible.

8 Conclusion

We built a Kahneman-inspired dual-model agent that routes elementary multiplication problems to the cheapest resource likely to solve them: a memory of past answers, a fast System 1 that answers what it is confident about, and a slow System 2 for the rest. On a 200-problem stream, confidence-framed routing alone matched a reasoning-only baseline at 43% of its cost, and adding memory made the full agent roughly $6.3\times$ cheaper and $6.5\times$ faster for a half-point accuracy cost. The same

mechanism that delivers those savings—memory replaying the first decision—also caps accuracy at the quality of first-contact routing. For an exploratory undergraduate project the takeaway is modest but clear: a small model, given explicit permission to defer, plus a simple memory, recovers most of a large model’s accuracy at a small fraction of its cost, and the interesting engineering lies in making that first routing decision good.

References

Lingjiao Chen, Matei Zaharia, and James Zou. FrugalGPT: How to use large language models while reducing cost and improving performance. *arXiv preprint arXiv:2305.05176*, 2023.

DeepSeek-AI. DeepSeek-R1: Incentivizing reasoning capability in LLMs via reinforcement learning. *arXiv preprint arXiv:2501.12948*, 2025.

Daniel Kahneman. *Thinking, Fast and Slow*. Farrar, Straus and Giroux, New York, 2011.

Mistral AI. Un ministral, des ministraux. <https://mistral.ai/news/ministraux>, 2024. Technical blog announcing the Ministral 3B and 8B models.

A Full Results Table

Table 2 reproduces the complete auto-generated summary with all recorded columns, rendered on a landscape page to fit its width.

Table 2: Per-configuration accuracy, latency, token and cost summary for the Thinking-Fast-and-Slow multiplication agent.

config	accuracy	accuracy_easy	accuracy_hard	n	n_easy	n_hard	mean_latency_s	total_latency_s	wall_clock_s	total_tokens	total_cost_usd	memory_hit_rate	escalation_rate	n_system2_calls	n_errors
pure_system1	0.920	1.000	0.810	200	116	84	0.53	105.25	9.69	26,113	0.0026	0.000	0.000	0	0
pure_system2	0.990	1.000	0.976	200	116	84	24.14	4828.68	440.20	237,570	0.5282	0.000	1.000	200	0
hybrid_no_memory	0.995	1.000	0.988	200	116	84	9.39	1878.78	201.79	152,579	0.2272	0.000	0.155	31	0
hybrid	0.985	1.000	0.964	200	116	84	3.74	747.13	747.15	57,644	0.0835	0.600	0.060	12	0

B Reproducibility

The full pipeline is reproducible from the repository root with three commands (dataset generation is seeded with seed 0):

```
python scripts/generate_dataset.py      # -> data/dataset.json
python scripts/run_experiments.py       # -> results/*.json   (calls the API)
python scripts/make_figures.py          # -> figures/*.png, results/summary_table.tex
```

The experiment scripts read an OpenRouter API key from the environment (`OPENROUTER_KEY`); a full run costs a few US dollars in OpenRouter credits. All four configurations reported here completed with zero errors.